112590024 邱紹溏 資料庫作業 HW2

4.24

正方形面積 = 2 × 2 = 4 圓形面積 = π × r² = π × 1² = π 圓/正方形 = π/4→ π = 4 × (落在圓內的點數) / (總點數)

```
kelen@laptop:~/文件/GitHub/OSHW/HW2/OSHW_2_individu
al_112590024/4_24$ go run count_pi.go
Number of points: 1000000
Points in circle: 785082
Estimated π = 3.140328
```

4_24.png

4.28

100 個 thread 各自拿到不同的 PID，sleep不同的時間，最後全部釋放，bitmap 歸零

```
kelen@laptop:~/文件/GitHub/OSHW/HW2/OSHW_2_indiv
$ gcc pid_ex.c
kelen@laptop:~/文件/GitHub/OSHW/HW2/OSHW_2_indiv
$ ./a.out
=== PID Manager - Multithreaded Test ===
PID range  : [300, 5000]  (4701 slots)
Threads    : 100
Max sleep  : 5 s

[thread   0] PID  300 acquired → sleeping 5 s
[thread   1] PID  301 acquired → sleeping 4 s
[thread   3] PID  302 acquired → sleeping 1 s
[thread   2] PID  303 acquired → sleeping 3 s
[thread   4] PID  304 acquired → sleeping 2 s
[thread   5] PID  305 acquired → sleeping 3 s
[thread   6] PID  306 acquired → sleeping 1 s
[thread   7] PID  307 acquired → sleeping 2 s
[thread   8] PID  308 acquired → sleeping 5 s
[thread   9] PID  309 acquired → sleeping 3 s
[thread  11] PID  310 acquired → sleeping 4 s
[thread  12] PID  311 acquired → sleeping 5 s
[thread  10] PID  312 acquired → sleeping 3 s
[thread  13] PID  313 acquired → sleeping 2 s
[thread  14] PID  314 acquired → sleeping 4 s
```

4_28_1.png

6.33

(a)

這個全域變數 available_resources

當多個 process/thread 同時讀寫它：

- Step 1: 從記憶體讀取 available_resources 的值
- Step 2: 計算新的值（加或減）
- Step 3: 把新值寫回記憶體

如果兩個 thread 同時做這三步，中間就會互相干擾。

```
17      int available_resources = MAX_RESOURCES;
```

a_issue.png

(b)

位置一：Thread A 讀完發現「夠用」，還沒減掉之前，Thread B 也讀了同一個值也覺得「夠用」，兩個都去減，結果 `available_resources` 變成負數。

```
if (available_resources < count)   // ← 讀取
    return -1;
else {
    available_resources -= count;  // ← 修改
    return 0;
}
```

6_33_b_problem.png

位置二：同樣的問題，兩個 thread 同時歸還資源，可能其中一個的修改被蓋掉。

```
available_resources += count;  // ← 讀取 + 計算
```

6_33_b_p2.png

(c)

用 Mutex 修正

```
int decrease_count(int count)
{
    pthread_mutex_lock(&mutex);
    //改成 while
    while (available_resources < count) {
        pthread_cond_wait(&cond, &mutex); /* atom
    }

    available_resources -= count;
    pthread_mutex_unlock(&mutex);
    return 0;
}
```

6_33_c_decrease_count.png

```c
//(c)用mutex lock 把它鎖住
int increase_count(int count)
{
    pthread_mutex_lock(&mutex);

    available_resources += count;
    printf("  [released] %d resource(s) returned, %d available\n",
           count, available_resources);

    // 打開全每個thread，每一個會重新確認 while
    pthread_cond_broadcast(&cond);

    pthread_mutex_unlock(&mutex);
    return 0;
}
```

6_33_c.png